

Construir una aplicación web donde un comercio pequeño atienda en un solo lugar los mensajes que le llegan por WhatsApp, Instagram, Messenger, Facebook y TikTok, tenga a sus clientes organizados, agende sus tareas y vea cómo va su equipo, sin pagar una plataforma cara ni depender de un proveedor de WhatsApp en particular.

MVP-CRM

Documentación técnica

Autor:

Samuel Pérez Serna

Septiembre 2026

Repositorio: github.com/SamuelPerezCO/MVP-CRM

Qué es MVP-CRM

MVP-CRM es una aplicación web para comercios que venden por chat. Junta en una sola pantalla los mensajes que llegan por WhatsApp, Instagram, Messenger, Facebook y TikTok, y al lado guarda la información del cliente que escribe: su nombre, su teléfono, por dónde llegó y qué se ha hablado con él. Desde esa misma pantalla el vendedor responde, se reparte las conversaciones con sus compañeros, les pone etiquetas y agenda tareas.

Está pensada para equipos de dos a diez personas. Se usa desde el navegador, en español, y se parece a las plataformas comerciales de este tipo (Treble, Leadsales, Kommo) pero sin cobro por agente.

El problema que resuelve

Un comercio que vende por mensajería suele atender desde el celular del vendedor. Cada red social tiene su propia app, así que hay que estar saltando entre cinco bandejas distintas. Cuando el vendedor está ocupado, un mensaje se queda sin responder y esa venta se pierde. Cuando otra persona del equipo retoma la conversación, no sabe qué se le había ofrecido al cliente. Y el dueño nunca tiene números: cuántos mensajes llegan, cuánto tardan en responderlos, quién atiende a quién.

Además, WhatsApp tiene una regla: si el cliente lleva más de 24 horas sin escribir, ya no se le puede mandar un mensaje libre, solo una plantilla aprobada por Meta, y cada plantilla cuesta dinero. Sin una herramienta que lleve esa cuenta, el comercio no sabe cuánto está gastando.



El problema en un vistazo: lo de abajo lo causa, lo de arriba es lo que se sufre.

Qué se quiso lograr

El objetivo general fue construir un CRM omnicanal para comercios: una sola bandeja para todos los canales, conectada con los clientes, el calendario, las plantillas de WhatsApp y las estadísticas, hecha con Django, htmx y PostgreSQL, y que pueda cambiar de proveedor de WhatsApp sin reescribir la aplicación.

Para llegar ahí se fijaron estos pasos:

- Entender qué necesita un comercio pequeño para atender y vender por chat, y escribirlo como requisitos.

- Diseñar la base de datos de modo que el cliente sea uno solo: el mismo en el chat, en el calendario y en las listas.
- Construir la pantalla principal y una forma sencilla de añadir secciones nuevas.
- Hacer la bandeja de entrada: filtros, hilo de chat, la regla de las 24 horas y las respuestas rápidas.
- Conectar WhatsApp a través de una pieza intercambiable, para poder usar un simulador en desarrollo y Meta en producción.
- Añadir la gestión de clientes, el calendario, las plantillas, el catálogo y las estadísticas.
- Escribir pruebas automáticas que protejan las reglas importantes.
- Publicarlo en internet sobre Vercel y Neon, con comandos para arrancar en limpio.

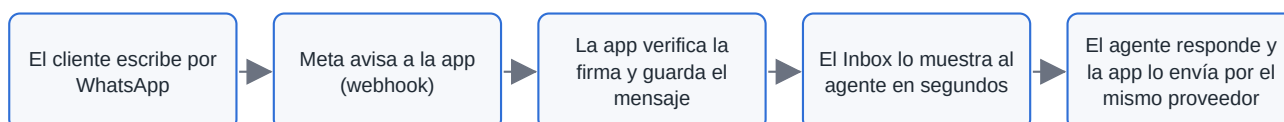
Qué hace la aplicación

Sección	Qué permite hacer
Inbox	Ver todas las conversaciones en una lista, filtrarlas por canal o por quién las atiende, abrir el chat, responder con texto o imágenes, usar respuestas rápidas, asignar la conversación a un compañero y ponerle etiquetas. Si pasaron más de 24 horas desde el último mensaje del cliente, solo deja enviar una plantilla.
Clientes (CRM)	Crear, buscar, editar y borrar clientes; ver su teléfono con la bandera del país, su correo y por qué canal llegó; exportar la lista a Excel; agrupar clientes en listas; escribirle a un cliente por primera vez desde su ficha.
Calendario	Agendar llamadas, entregas o recordatorios, cada uno con un cliente y un responsable, en la hora de Colombia.
Equipo	El usuario maestro crea, edita y desactiva a los demás usuarios desde la aplicación. Nadie se borra: su historial se conserva.
Plantillas de WhatsApp	Escribir plantillas con encabezado, cuerpo con variables, pie y botones; ver cómo quedarán; mandarlas a Meta para aprobación y consultar si fueron aceptadas o rechazadas. La app calcula cuánto cuesta cada envío.
Mi comercio	Llevar el catálogo de productos con precio, categoría, marca y estado, uno a uno o importados.
Estadísticas	Ver cuántos mensajes llegan por canal y por hora, cuánto tarda el equipo en responder, cuánto trabaja cada agente y qué tipo de clientes escriben.
Embudos y Campañas	Existen como pantallas iniciales para organizar el proceso de venta y las campañas; todavía no tienen funciones completas.

Las secciones de la aplicación, en el orden de la barra lateral.

Cómo funciona por dentro

La aplicación tiene un servidor escrito en Python con Django y una interfaz que se actualiza por partes gracias a htmx, sin necesidad de compilar nada del lado del navegador. Todo ocurre en una sola página: al hacer clic en un icono de la barra lateral, el servidor devuelve solo el trozo de pantalla que cambia.



El camino de un mensaje, desde que el cliente escribe hasta que el vendedor responde.

Las piezas principales son estas:

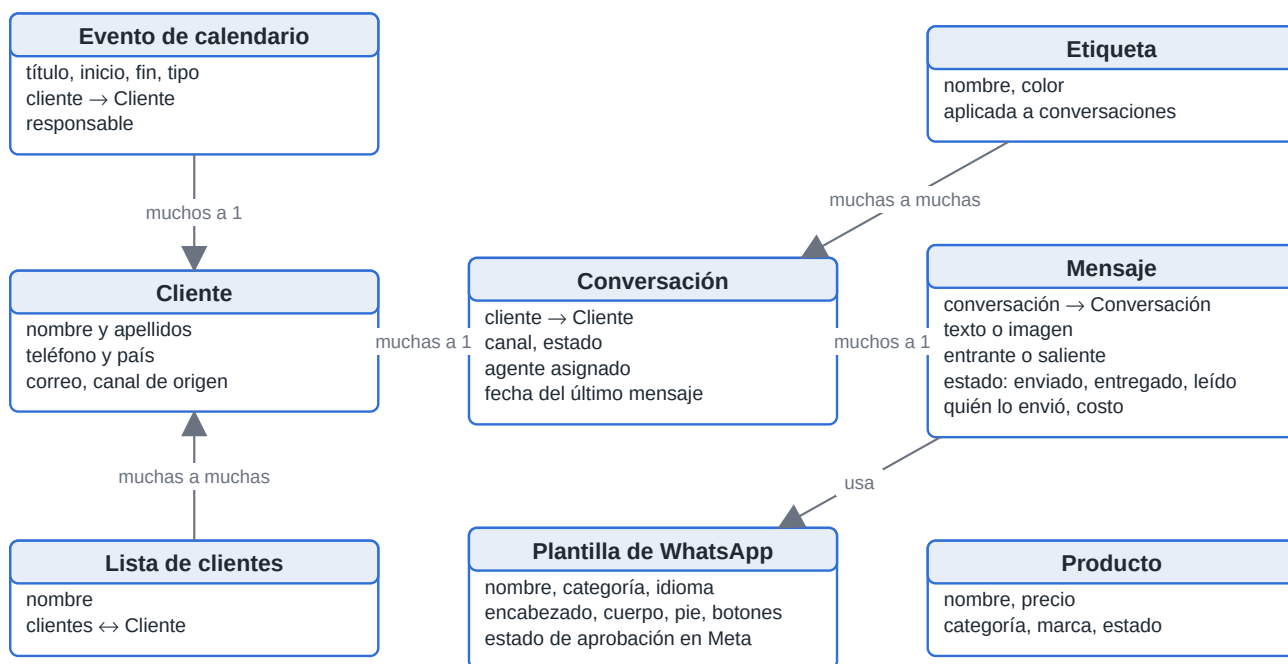
- **Proveedores de mensajería.** Toda la conexión con WhatsApp está encerrada en una pieza que se elige con una sola variable de configuración: fake (un simulador para desarrollar sin credenciales) o

meta. Cambiar de proveedor no toca pantallas, modelos ni reglas.

- **Webhook seguro.** Cuando el proveedor avisa de un mensaje nuevo, la app comprueba primero que el aviso venga firmado por él. Si el mismo aviso llega dos veces, el mensaje se guarda una sola vez.
- **Un solo cliente.** La persona que escribe en el chat es la misma fila de la tabla Clientes, la misma del calendario y la misma de las listas. No hay copias.
- **Agentes.** Las cuentas del equipo se definen en la configuración del servidor (con contraseña cifrada) o las crea el usuario maestro desde la app. Cada mensaje enviado guarda quién lo escribió.
- **Base de datos.** SQLite mientras se desarrolla en el computador; PostgreSQL en Neon cuando está publicada. Las imágenes que llegan por chat se guardan en Vercel Blob.
- **Publicación.** Corre en Vercel. Todo lo que cambia entre el computador y producción (base de datos, proveedor, credenciales, dominio) va en variables de entorno documentadas en `.env.example`.
- **Pruebas.** Más de veinte módulos de pruebas automáticas cubren las reglas de negocio, los webhooks de cada proveedor y las pantallas.

Cómo guarda los datos

Todo gira alrededor del cliente. Cada cliente puede tener varias conversaciones (una por canal), cada conversación tiene muchos mensajes, y a las conversaciones se les ponen etiquetas. Los eventos del calendario y las listas apuntan al mismo cliente. Las plantillas y los productos son tablas aparte.



Las tablas principales y cómo se relacionan. La flecha apunta a la tabla de la que depende.

Requisitos

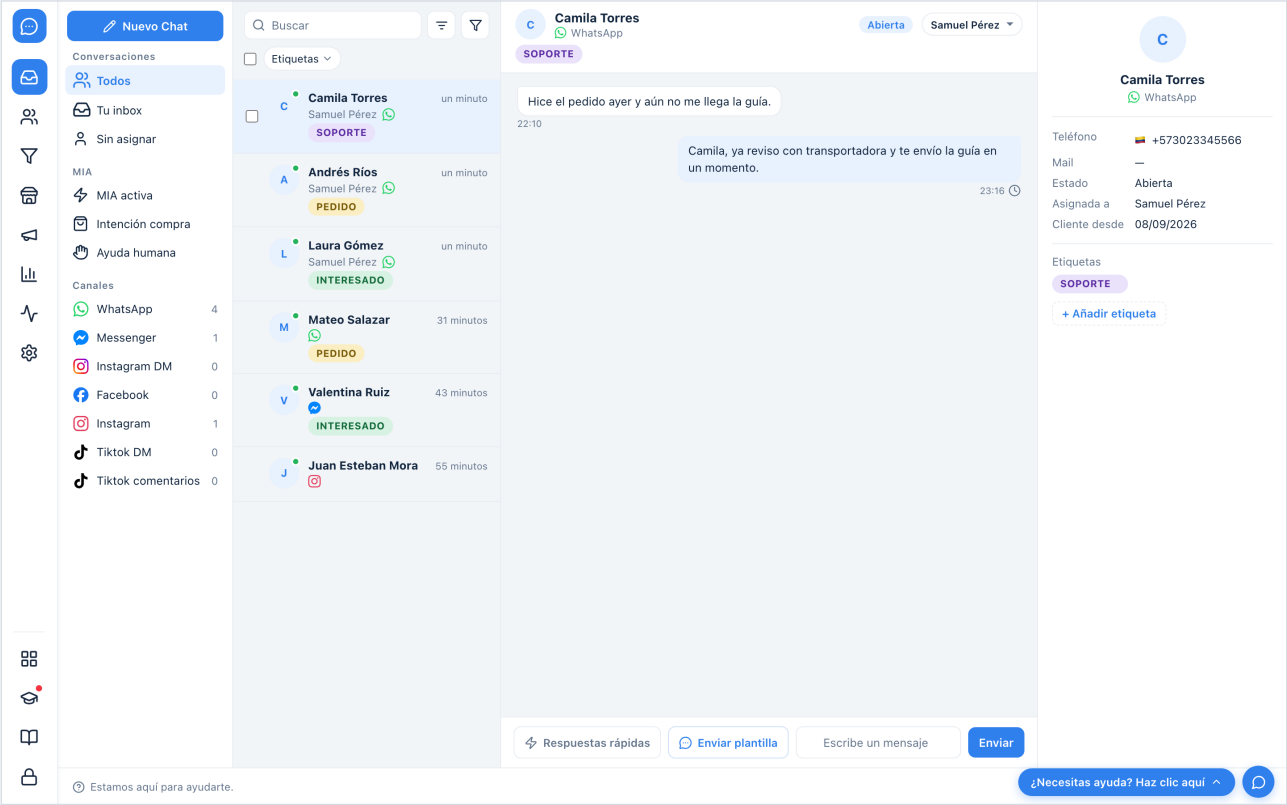
Lo que la aplicación debe hacer (requisitos funcionales) y cómo debe comportarse (requisitos no funcionales), resumido.

ID	Requisito	En pocas palabras
RF-01	Inicio de sesión	Nadie entra sin una cuenta de agente.
RF-02	Bandeja unificada	Todos los canales en una lista, con filtros por canal y por responsable.

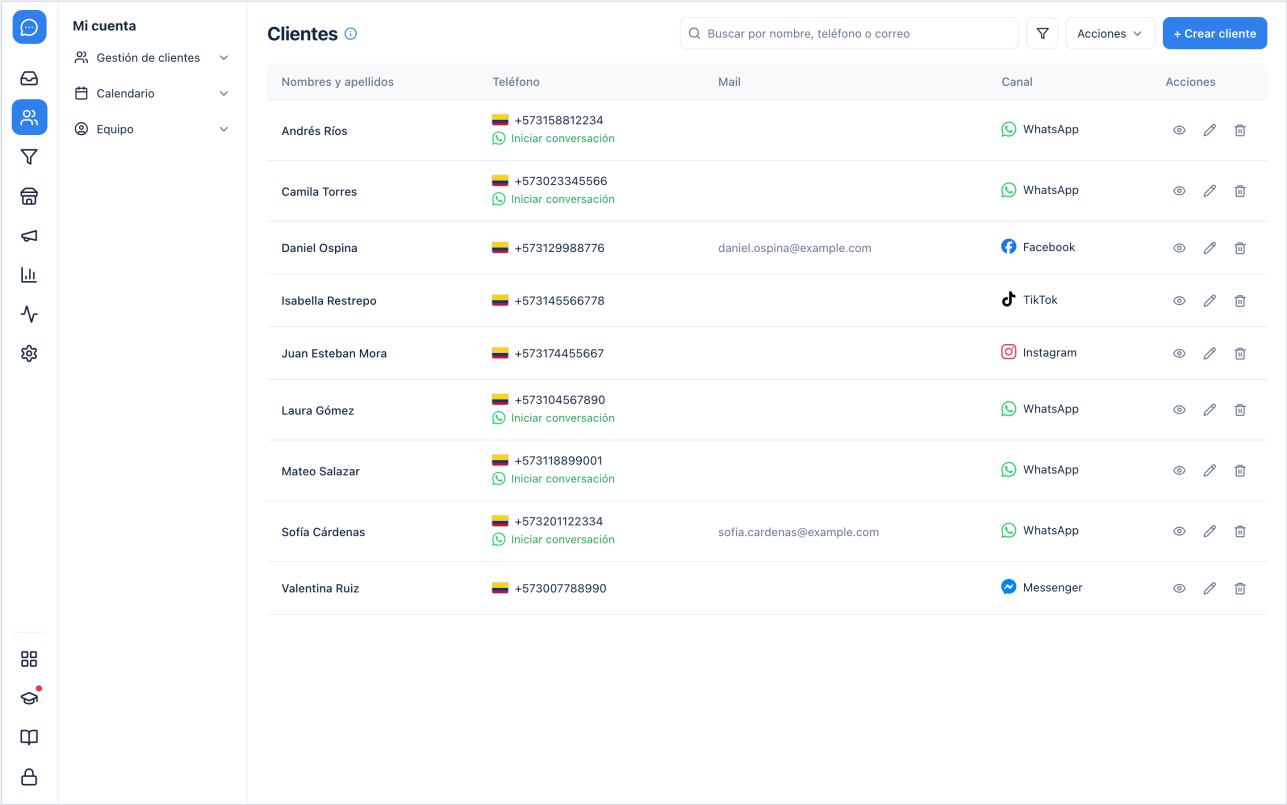
ID	Requisito	En pocas palabras
RF-03	Recibir y enviar	Los mensajes entran por el webhook del proveedor y las respuestas salen por el mismo proveedor, con su estado de entrega.
RF-04	Ventana de 24 horas	Pasadas 24 horas sin respuesta del cliente, solo se pueden enviar plantillas.
RF-05	Asignar y etiquetar	Cada conversación tiene un responsable y etiquetas.
RF-06	Clientes	Crear, editar, buscar, borrar, exportar a Excel y agrupar en listas.
RF-07	Calendario	Eventos con cliente y responsable.
RF-08	Plantillas	Crear, previsualizar, enviar a Meta y saber su costo.
RF-09	Estadísticas	Volumen, tiempos de respuesta, rendimiento por agente.
RF-10	Equipo	El usuario maestro administra a los demás usuarios.
RNF-01	Seguridad	Sesión obligatoria, webhooks firmados, contraseñas cifradas, simulador apagado en producción.
RNF-02	Proveedor intercambiable	Cambiar de WhatsApp simulado a Meta es cambiar una variable.
RNF-03	Sin servidor propio	Corre en Vercel con base de datos en Neon y archivos en Vercel Blob.
RNF-04	Fácil de usar	Una sola página, en español, que se actualiza sola.
RNF-05	Fácil de mantener	Añadir una sección es una línea; las reglas tienen pruebas automáticas.

Requisitos funcionales (RF) y no funcionales (RNF).

Pantallas



Inbox: la lista de conversaciones a la izquierda, el chat en el centro y la ficha del cliente a la derecha.



Clientes: la tabla con teléfono, correo, canal y acciones.

[illegible]

Calendario: la semana con los eventos y el cliente de cada uno.

Widget de WhatsApp

Plantillas de WhatsApp

Respuestas rápidas

Mensajes de bienvenida

Temas de conversación

Reglas de mensajería

Asignación automática

Buscar por nombre

Sincronizar con WhatsApp

Crear plantilla +

Todas	Pendientes	Aceptadas	Rechazadas	Desactivadas		
Nombre	Tipo	Categoría	Texto	Equipo	Activo	Estado
bienvenida_tienda	Mensaje personalizado	Marketing	¡Hola {{1}}! Gracias por escribirnos a {{2}}. ¿En qué podemos...		Sí	Pendiente sin enviar a WhatsApp
confirmacion_pedido	Mensaje personalizado	Utility	Hola {{1}}, tu pedido #{{2}} fue confirmado y sale mañana.		Sí	Pendiente sin enviar a WhatsApp
recordatorio_pago	Mensaje personalizado	Utility	Hola {{1}}, te recordamos que el pago de tu pedido vence el...		Sí	Pendiente sin enviar a WhatsApp

Plantillas de WhatsApp: una pestaña por estado de aprobación.

Cómo se ejecuta

En un computador, con Python 3.12 instalado, basta con crear un entorno virtual, instalar las dependencias, copiar `.env.example` a `.env`, aplicar las migraciones y arrancar el servidor:

```
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python manage.py migrate
python manage.py runserver
```

El `.env` de ejemplo ya trae el proveedor simulado, así que se puede probar todo sin credenciales de WhatsApp. Para producción se define `MESSAGING_PROVIDER=meta`, la base de datos de Neon y las credenciales del proveedor, y Vercel hace el resto. Antes de conectar el número real, el comando `go_live` deja la aplicación vacía y conserva solo al equipo.

Las pruebas se corren con `python manage.py test`.

Bibliografía

- Django Software Foundation (2026) *Django documentation, release 6.1*. <https://docs.djangoproject.com/en/6.1/>
- Big Sky Software (2026) *htmx documentation*. <https://htmx.org/docs/>
- Meta Platforms (2026) *WhatsApp Business Platform: Cloud API y pricing*. <https://developers.facebook.com/docs/whatsapp/>
- Vercel (2026) *Vercel documentation*. <https://vercel.com/docs> · Neon (2026) *Neon documentation*. <https://neon.com/docs>